
Compte Rendu de TP 2 : Pipelines agiles, versionnement et orchestration

Module : MLOps & DataOps

Youssef Sarraf

Master d'Excellence en Intelligence Artificielle (MEIA)
Faculté des Sciences Ben M'Sik (FSBM), Université Hassan II

31 mai 2026

Résumé

Ce document présente les livrables réalisés dans le cadre du TP 2. L'objectif était de transformer un traitement de données artisanal en un pipeline industrialisé utilisant la Modern Data Stack : DuckDB, dbt, Dagster et GitHub Actions.

Table des matières

1	Historique Git du Projet	2
2	Schéma du Pipeline Industrialisé	2
3	Réponses aux Questions d'Analyse	3
4	Captures d'Écran des Commandes Importantes	4
4.1	Étape de Validation Minimale (Fail Fast)	4
4.2	Transformation des Données avec dbt	4
4.3	Orchestration globale avec Dagster	5

1 Historique Git du Projet

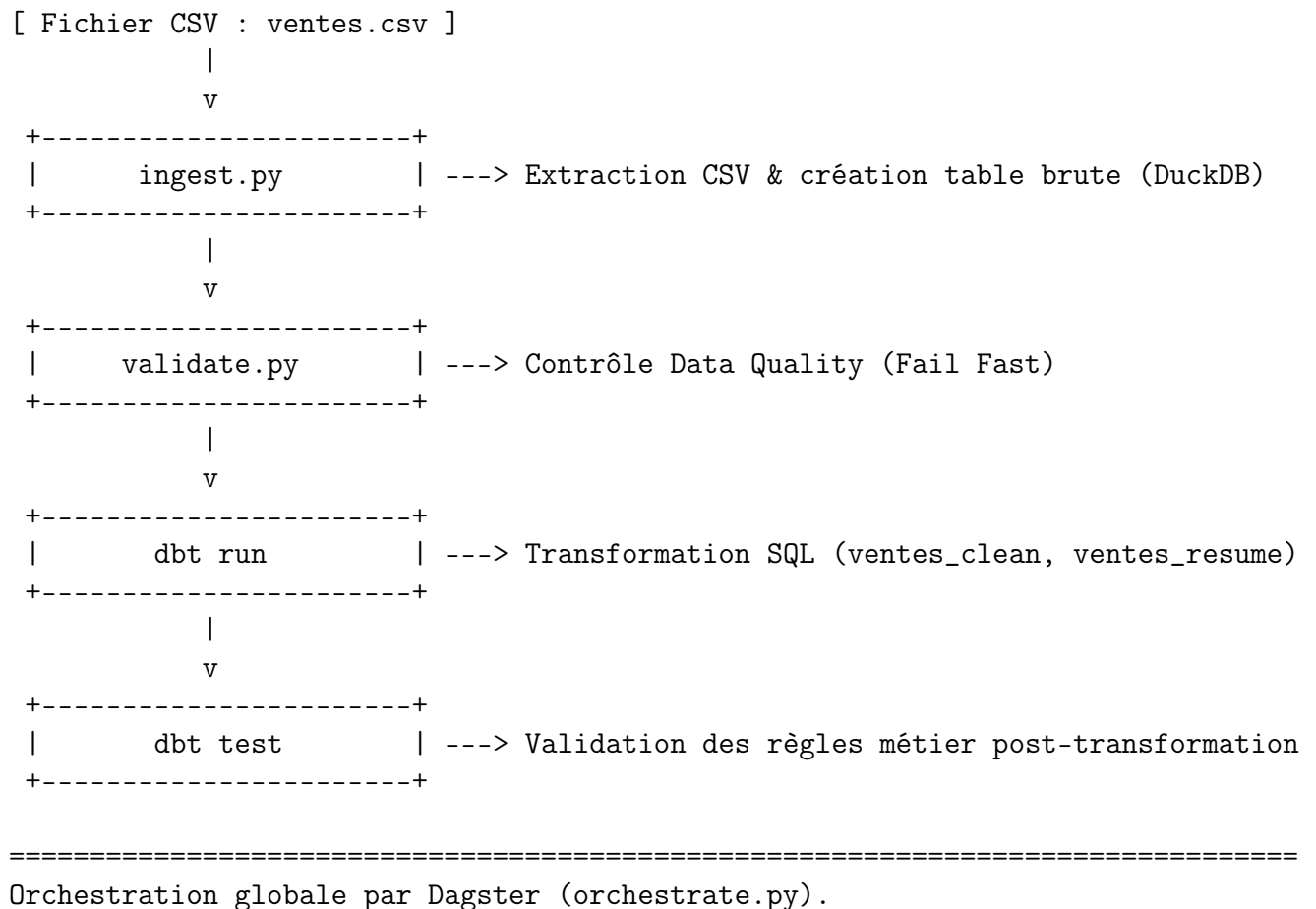
L'ensemble du projet a été versionné dès son initialisation. Les différentes étapes (ingestion, validation, transformation, orchestration et CI) ont fait l'objet de commits distincts pour assurer une traçabilité complète. La figure ci-dessous illustre l'historique des modifications sur la branche principale.

```
youssef@DELL:~/Documents$ cd ~/Documents/tp_chapitre2_pipeline
youssef@DELL:~/Documents/tp_chapitre2_pipeline$ git log --oneline --graph --all
* b69a849 (HEAD -> main) Correction des paramètres de l'orchestrateur Dagster
* e7ff5d3 Ajout d'une CI simple
* 3ebcfb9 Ajout de l'orchestration Dagster
* 2c56613 Ajout des tests simples dbt
* 9f881ec Ajout des modèles dbt
* 8dd62a8 Ajout de l'étape de validation minimale
* 44d016b Ajout de l'étape d'ingestion
* 23e8239 Initialisation du projet pipeline
```

FIGURE 1 – Historique des commits Git du projet dans le terminal

2 Schéma du Pipeline Industrialisé

Le flux de données a été structuré de manière modulaire, chaque étape ayant une responsabilité unique au sein de "l'usine à données".



3 Réponses aux Questions d'Analyse

1. **En quoi ce pipeline est-il plus structuré qu'un notebook unique ?**
Il sépare clairement les responsabilités en modules distincts (ingestion, validation, transformation). Contrairement à un notebook qui mélange code, exécution séquentielle manuelle et données, cette structure modulaire permet d'automatiser l'enchaînement, d'isoler les erreurs rapidement et de versionner chaque étape indépendamment.
2. **Quel est le rôle de Git dans ce projet ?**
Git assure le versionnement et la traçabilité du code source. Il permet de conserver un historique précis des modifications et garantit la capacité de revenir à une version précédente fonctionnelle en cas d'anomalie.
3. **Quel est le rôle de `requirements.txt` ?**
Ce fichier fige les versions exactes des dépendances utilisées (pandas, duckdb, dbt, dagster). Il garantit la reproductibilité de l'environnement : tout développeur clonant le projet pourra installer exactement les mêmes outils sans conflit de versions.
4. **Pourquoi ajouter une étape `validate.py` ?**
Cette étape sert de contrôle qualité en amont. Elle applique le principe de "Fail Fast" : le pipeline est stoppé immédiatement si la donnée entrante est corrompue (valeurs nulles ou colonnes manquantes), évitant ainsi de polluer les bases de données cibles.
5. **Quel est le rôle de `dbt test` ?**
Tandis que `validate.py` s'assure de la qualité des données brutes, `dbt test` valide la qualité des données *transformées* grâce aux règles métier définies dans `schema.yml`. Cela certifie que la logique SQL n'a pas introduit d'erreurs.
6. **Qu'apporte Dagster par rapport à une exécution manuelle ?**
Dagster apporte l'orchestration formelle du flux. Il remplace l'exécution manuelle script par script en définissant un graphe dirigé acyclique (DAG) des dépendances, et offre une interface visuelle pour déclencher, surveiller et relancer les tâches automatisées.
7. **Qu'apporte la CI ?**
L'Intégration Continue (via GitHub Actions) permet d'exécuter des vérifications automatisées (ex. syntaxe Python) de manière asynchrone à chaque push. Elle prévient le déploiement de code défectueux sur la branche principale (`main`).
8. **Que manque-t-il encore pour un pipeline pleinement industrialisé ?**
Pour un système complet en production, il manque le versionnement des données entrantes (ex. DVC), le déploiement sur une infrastructure Cloud (ex. Data Warehouse distant au lieu de DuckDB local), ainsi que le volet MLOps (entraînement de modèle, model registry et monitoring des performances algorithmiques).

4 Captures d'Écran des Commandes Importantes

4.1 Étape de Validation Minimale (Fail Fast)

Preuve du blocage volontaire du pipeline suite à la détection de la ligne incomplète dans le fichier source `ventes.csv`.

```
(.venv) youssef@DELL:~/Documents/tp_chapitre2_pipeline$ cat > dbt_pipeline/dbt_project.yml << (.venv) youssef@DELL:~/Documents/tp_chapitre2_pipeline$ cat > pipeline/validate.py << 'EOF'
import duckdb

db_path = "ventes.duckdb"
required_columns = ["date", "produit", "categorie", "quantite", "prix_unitaire", "ville"]

con = duckdb.connect(db_path)
columns = con.execute("DESCRIBE ventes_raw").fetchall()
existing_columns = {col[0] for col in columns}
missing = required_columns - existing_columns

if missing:
    raise ValueError(f"Colonnes manquantes: {missing}")

null_count = con.execute("""
    SELECT COUNT(*) FROM ventes_raw
    WHERE produit IS NULL OR quantite IS NULL OR prix_unitaire IS NULL
""").fetchone()[0]
con.close()

if null_count > 0:
    raise ValueError(f"Données invalides : {null_count} lignes incomplètes")
else:
    print("Validation réussie : schéma et qualité minimale OK")
EOF

python pipeline/validate.py || true
git add pipeline/validate.py
git commit -m "Ajout de l'étape de validation minimale"
Traceback (most recent call last):
  File "/home/youssef/Documents/tp_chapitre2_pipeline/pipeline/validate.py", line 21, in <module>
    raise ValueError(f"Données invalides : {null_count} lignes incomplètes")
ValueError: Données invalides : 1 lignes incomplètes
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    data/

nothing added to commit but untracked files present (use "git add" to track)
```

FIGURE 2 – Déclenchement de la `ValueError` par `validate.py`

4.2 Transformation des Données avec dbt

Preuve de la connexion réussie à DuckDB et de la compilation des modèles SQL sans erreur.

```
16:04:21 All checks passed!
16:04:24 Running with dbt=1.11.11
16:04:24 Registered adapter: duckdb=1.10.1
16:04:24 Unable to do partial parsing because saved manifest not found. Starting full parse.
16:04:26 Found 2 models, 485 macros
16:04:26
16:04:26 Concurrency: 1 threads (target='dev')
16:04:26
16:04:26 1 of 2 START sql table model main.ventes_clean ..... [RUN]
16:04:26 1 of 2 OK created sql table model main.ventes_clean ..... [OK in 0.08s]
16:04:26 2 of 2 START sql table model main.ventes_resume ..... [RUN]
16:04:26 2 of 2 OK created sql table model main.ventes_resume ..... [OK in 0.03s]
16:04:26
16:04:26 Finished running 2 table models in 0 hours 0 minutes and 0.23 seconds (0.23s).
16:04:26
16:04:26 Completed successfully
16:04:26
16:04:26 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
[main 9f881ec] Ajout des modèles dbt
6 files changed, 318 insertions(+)
create mode 100644 dbt_pipeline/.user.yml
create mode 100644 dbt_pipeline/dbt_project.yml
create mode 100644 dbt_pipeline/logs/dbt.log
create mode 100644 dbt_pipeline/models/ventes_clean.sql
create mode 100644 dbt_pipeline/models/ventes_resume.sql
create mode 100644 dbt_pipeline/profiles.yml
```

FIGURE 3 – Exécution de `dbt run` réussie

4.3 Orchestration globale avec Dagster

Interface visuelle de Dagster démontrant l'enchaînement automatisé et le succès des différentes étapes (*op*) du graphe `ventes_pipeline`.

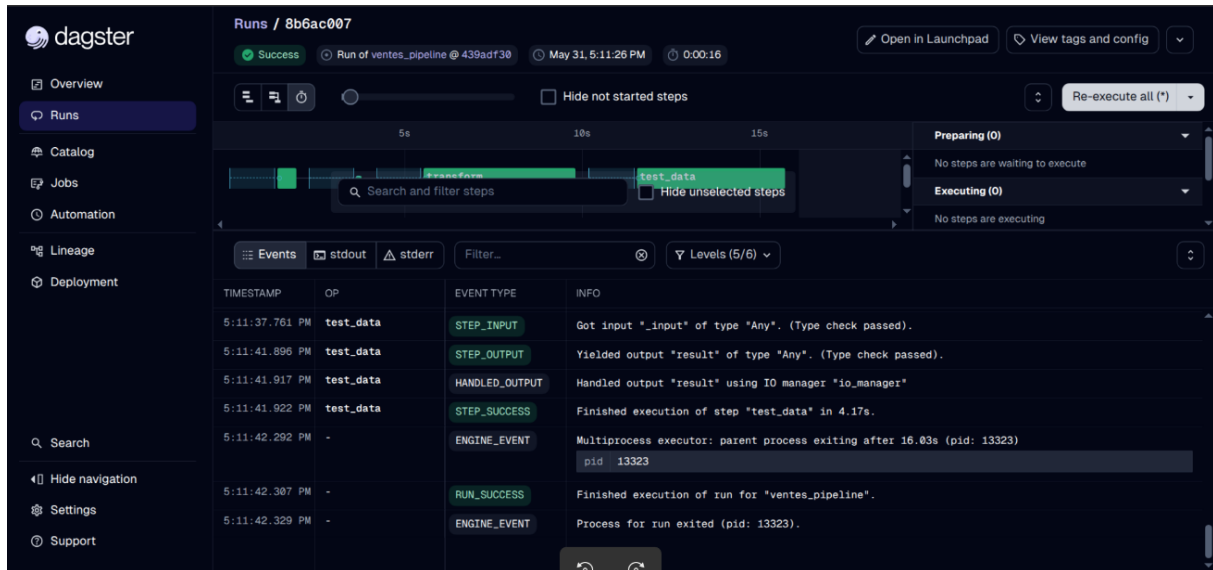


FIGURE 4 – Succès du run dans l'interface Dagster UI